λ_S^2 — Conditionals and Selection:

Extending λ_S^1 with Enum Returns and Protocol Branching

Alexandre Zua Caldeira
Independent Researcher
azuacaldeira@gmail.com

March 2026

Abstract

λ_S^1 [1] established that base-type return values are orthogonal to the session discipline. λ_S^2 breaks this orthogonality: methods may return *enum values* that determine which branch of a *selection* ($\oplus$) type the protocol follows. We add selection types, enum return types, and a **match** expression to the calculus. The key typing rule (T-Match) checks each branch under a different successor state, reflecting the internal choice made by the callee. We prove preservation, progress, and session completion, and show that the T-Match rule corresponds exactly to the `SelectionSwitch` mechanism in BICA Reborn's `TypestateChecker`.

1 Introduction

In λ_S^1, a method $x.m()$ returns a base-type value that the client can bind but not inspect. This models methods like `read()` $\rightarrow$ `String` or `checkBalance()` $\rightarrow$ `double` where the return value carries data but not protocol information.

Real protocols, however, have *internal choice*: the callee decides which branch to take and communicates the decision to the caller via the return value. In Java:

```
HasNextResult r = iter.hasNext();  // returns TRUE or FALSE
switch (r) {
    case TRUE  -> { iter.next(); /* ... */ }
    case FALSE -> { /* done */ }
}
```

The return type `HasNextResult` is not a scalar—it is an *enum whose constants correspond to selection labels.* The client *must* branch on the return value, and each branch faces a different continuation of the protocol.

λ_S^2 formalises this pattern. It extends λ_S^1 with:

1. **Selection types**: $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$.
2. **Enum types**: $\mathbf{Enum}(l_1, \ldots, l_n)$ with constants corresponding to selection labels.
3. **Match expression**: $\mathbf{match}\ x\ \mathbf{with}\ \{l_i \Rightarrow e_i\}_i$ that branches on an enum value.
4. **A modified T-Call** that, at a selection state, returns an enum value and does *not* advance the state (the state advances inside each match branch).

Breaking orthogonality. In λ_S^1, the return type τ never affects Γ. In λ_S^2, the return value *determines* how Γ evolves: if the callee returns l_i, the object moves to state s_i. This is the fundamental difference.

Structure. Section 2 recalls $\lambda_S^{0,1}$. Section 3 introduces the new syntax. Section 4 presents the type system (eight rules). Section 5 gives the operational semantics. Section 6 proves soundness. Section 7 works through the Iterator example. Section 8 discusses the result.

2 $\lambda_S^{0,1}$ Recap

λ_S^0 formalises straight-line typestate checking with five rules; all methods return **Unit**. λ_S^1 adds base-type returns and proves they are orthogonal to the session discipline (Orthogonality Theorem). Session types in both calculi use only $\&\{\ldots\}$ (branch) and **end**.

Key judgement form. $\mathcal{CT};\ \Gamma \vdash e : T \dashv \Gamma'$, where Γ tracks session-typed variables $x : C \mathbin{@} s$.

Key rule (T-Call^1).

$$\textsc{T-Call}^1\ \frac{\Gamma(x) = C \mathbin{@} s \qquad \delta(s, m) = s' \qquad msig(C, m) = \tau}{\mathcal{CT};\ \Gamma \vdash x.m() : \tau \dashv \Gamma[x \mapsto C \mathbin{@} s']}$$

The transition $\delta(s, m) = s'$ is deterministic: there is exactly one successor state regardless of what τ is.

3 λ_S^2 Syntax

3.1 Selection Types

Session types now include selection (internal choice):

$$S \ ::= \ \&\{m_1 : S_1, \ldots, m_n : S_n\} \ \mid \ \oplus\{l_1 : S_1, \ldots, l_n : S_n\} \ \mid \ \mathbf{end}$$

A selection state s has transitions $\delta_\oplus(s, l_i) = s_i$ for each label l_i. These are *not* callable methods: the callee (object) makes the choice, not the caller (client).

Definition 3.1 (Pure Selection State)**.** A state s is a *pure selection state* if $enabledMethods(s) = \emptyset$ and $enabledSelections(s) \neq \emptyset$.

3.2 Enum Types

Definition 3.2 (Enum Type)**.** An enum type $\mathbf{Enum}(l_1, \ldots, l_n)$ has constants $l_1, \ldots, l_n$ that correspond exactly to the labels of a selection type.

The extended type grammar:

$$T \ ::= \ \mathbf{Unit} \ \mid \ C \ \mid \ \tau \ \mid \ \mathbf{Enum}(l_1, \ldots, l_n)$$

3.3 Match Expression

Values and expressions extend:

$$\begin{aligned} v \ &::= \ x \ \mid \ \mathbf{skip} \ \mid \ c \ \mid \ l && (l = \text{enum constant}) \\ e \ &::= \ \ldots \ \mid \ \mathbf{match}\ x\ \mathbf{with}\ \{l_i \Rightarrow e_i\}_{i \in I} \end{aligned}$$

The match expression inspects an enum-typed variable and branches. Each branch e_i is checked under a different successor state.

3.4 Selection-Returning Methods

When a method m of class C sits before a selection state—i.e., $\delta(s, m)$ leads to a pure selection state $s_\oplus$ with labels $\{l_1, \ldots, l_n\}$—then $msig(C, m) = \mathbf{Enum}(l_1, \ldots, l_n)$. The enum constant returned by the method tells the client which selection label was chosen.

4 Type System

Eight rules. Rules marked ($\star$) are new; ($\star\star$) is significantly modified.

Unchanged rules. T-Skip, T-Const, T-Var, T-New, T-Let are identical to λ_S^1.

$$\textsc{T-Label}\ (\star)\ \frac{l \in \{l_1, \ldots, l_n\}}{\mathcal{CT};\ \Gamma \vdash l : \mathbf{Enum}(l_1, \ldots, l_n) \dashv \Gamma}$$

T-Label types an enum constant. Environment unchanged.

$$\textsc{T-Call}\ (\star\star)\ \frac{\Gamma(x) = C \mathbin{@} s \qquad \delta(s, m) = s' \qquad msig(C, m) = T_{ret}}{\mathcal{CT};\ \Gamma \vdash x.m() : T_{ret} \dashv \Gamma[x \mapsto C \mathbin{@} s']}$$

T-Call is the same as λ_S^1 syntactically. The key difference is that s' may now be a *pure selection state*. When it is, $T_{ret} = \mathbf{Enum}(l_1, \ldots, l_n)$ and the client is obliged to branch on the return value using T-Match.

$$\textsc{T-Match}\ (\star)\ \frac{\begin{array}{c} x : \mathbf{Enum}(l_1, \ldots, l_n) \qquad \Gamma(y) = C \mathbin{@} s_\oplus \\ s_\oplus \text{ is a pure selection state} \qquad \forall\, i.\ \delta_\oplus(s_\oplus, l_i) = s_i \qquad \forall\, i.\ \mathcal{CT};\ \Gamma[y \mapsto C \mathbin{@} s_i] \vdash e_i : T \dashv \Gamma'_i \end{array}}{\mathcal{CT};\ \Gamma \vdash \mathbf{match}\ x\ \mathbf{with}\ \{l_i \Rightarrow e_i\}_i : T \dashv \prod_i \Gamma'_i}$$

T-Match is the core new rule. It:

1. Requires the scrutinee x to have an enum type matching the selection labels at $s_\oplus$.
2. Checks each branch e_i with the session variable y advanced to the selection successor s_i.
3. All branches must produce the same result type T.
4. The output environments Γ'_i are *joined*: each session variable must end at the same state in all branches (or be at $\bot$ in all branches). We write $\prod_i \Gamma'_i$ for this merge, which fails if branches disagree.

Remark 4.1 (Environment merging)**.** $\prod_i \Gamma'_i$ requires that for each session-typed variable $z : C \mathbin{@} s^{(i)}$ across branches, all $s^{(i)}$ are equal. This ensures that after the match, the session state is determinate.

5 Operational Semantics

Configurations extend with enum constants: $v ::= \ldots \mid l$.

$$\textsc{R-Call-Sel}\ (\star)\ \frac{H(\ell) = C \qquad \sigma(\ell) = s \qquad \delta(s, m) = s_\oplus \qquad s_\oplus \text{ is selection} \qquad \delta_\oplus(s_\oplus, l_j) = s_j}{\langle H,\ \sigma,\ \ell.m()\rangle \longrightarrow \langle H,\ \sigma[\ell \mapsto s_j],\ l_j\rangle}$$

R-Call-Sel: when a method call leads to a selection state, the object *nondeterministically* chooses a label l_j, advances to s_j, and returns the label to the caller. This models the callee's internal decision.

$$\text{R-Match } (\star) \quad \frac{}{\langle H,\ \sigma,\ \textbf{match}\ l_j\ \textbf{with}\ \{l_i \Rightarrow e_i\}_i\rangle \longrightarrow \langle H,\ \sigma,\ e_j\rangle}$$

R-Match: evaluate to the branch matching the enum constant.
R-New, R-Call (non-selection), R-Let-Ctx, R-Let-Val are unchanged from λ_S^1.

Remark 5.1. In R-Call-Sel, the session store is updated to s_j (the post-selection state), not $s_\oplus$. This corresponds to BICA Reborn's `advancePastSelections`: the checker auto-advances past the pure selection state to the branch target.

6 Soundness

6.1 Conservative Extension

Theorem 6.1 (Conservative Extension)**.** *λ_S^2 is a conservative extension of λ_S^1: when no selection types appear in $\mathcal{CT}$ and no match expressions appear in e, every λ_S^1 derivation is valid in λ_S^2.*

Proof. T-Call reduces to the λ_S^1 version when s' is not a selection state. T-Label and T-Match are never triggered. □

6.2 Preservation

Theorem 6.2 (Preservation)**.** *If $\mathcal{CT} \vdash \langle H,\ \sigma,\ e\rangle : T \dashv \Gamma'$ and $\langle H,\ \sigma,\ e\rangle \longrightarrow \langle H_1,\ \sigma_1,\ e_1\rangle$, then there exist T' and Γ_1' such that $\mathcal{CT} \vdash \langle H_1,\ \sigma_1,\ e_1\rangle : T' \dashv \Gamma_1'$.*

Proof. New cases:

- R-Call-Sel: by typing, $\sigma[\ell \mapsto s_j]$ and result $l_j : \textbf{Enum}(\ldots)$. The runtime environment has ℓ at s_j. By T-Label, l_j is typeable.
- R-Match: by typing, each branch e_j is typeable under $\Gamma[y \mapsto C \mathbin{@} s_j]$. After the match resolves to e_j, the typing matches the j-th branch premise.
- Other cases: identical to λ_S^1. □

6.3 Progress

Theorem 6.3 (Progress)**.** *Well-typed non-values can always reduce.*

Proof. New cases:

- $e = \ell.m()$ at selection state: R-Call-Sel applies (selection labels are always non-empty).
- $e = \textbf{match}\ l_j\ \textbf{with}\ \ldots$: R-Match applies (the match is exhaustive by T-Match). □

6.4 Session Completion

Corollary 6.4 (Session Completion)**.** *Terminated λ_S^2 programs have all locations at $\bot$.*

Proof. By iterated preservation. The environment-merge condition ($\bigsqcap_i \Gamma_i'$) in T-Match ensures that all branches leave session variables in the same state, so session completion is well-defined regardless of which branch executes. □

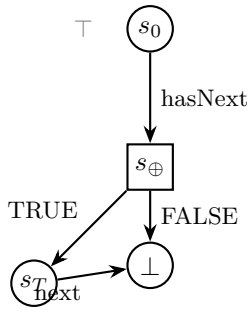

Figure 1: Iterator state space. Diamond = pure selection state (internal choice). The client cannot call methods at $s_\oplus$; it must match on the enum return of `hasNext()`.

7 Worked Examples

7.1 Iterator: `hasNext` → Selection

Session type:

$$S_{\text{Iter}} = \&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : \mathbf{end}\},\ \mathit{FALSE} : \mathbf{end}\}\}$$

(Without recursion, for λ_S^2's restriction.)
Method signatures: $\mathit{msig}(\text{Iter}, \mathit{hasNext}) = \mathbf{Enum}(\mathit{TRUE}, \mathit{FALSE})$, $\mathit{msig}(\text{Iter}, \mathit{next}) = \mathbf{Unit}$.
Client program:

$$\begin{aligned} e_{\text{iter}} = {} & \mathbf{let}\ \mathit{it} = \mathbf{new}\ \text{Iter}()\ \mathbf{in} \\ & \mathbf{let}\ r = \mathit{it}.\mathit{hasNext}()\ \mathbf{in} \\ & \mathbf{match}\ r\ \mathbf{with}\ \{ \\ & \quad \mathit{TRUE} \Rightarrow \mathbf{let}\ _ = \mathit{it}.\mathit{next}()\ \mathbf{in}\ \mathbf{skip}, \\ & \quad \mathit{FALSE} \Rightarrow \mathbf{skip}\ \} \end{aligned}$$

Typing derivation.

1. T-New: $\Gamma_0 = \{\mathit{it} : \text{Iter} \mathbin{@} s_0\}$.
2. T-Call: $\delta(s_0, \mathit{hasNext}) = s_\oplus$. Returns $\mathbf{Enum}(\mathit{TRUE}, \mathit{FALSE})$. $\Gamma_1 = \{\mathit{it} : \text{Iter} \mathbin{@} s_\oplus\}$.
3. T-Match: $s_\oplus$ is pure selection.
   - TRUE branch: $\delta_\oplus(s_\oplus, \mathit{TRUE}) = s_T$. Check e_T under $\Gamma[\mathit{it} \mapsto s_T]$. T-Call: $\delta(s_T, \mathit{next}) = s_\bot$. $\Gamma'_T = \{\mathit{it} \mathbin{@} s_\bot\}$.
   - FALSE branch: $\delta_\oplus(s_\oplus, \mathit{FALSE}) = s_F = s_\bot$. $\Gamma'_F = \{\mathit{it} \mathbin{@} s_\bot\}$.
4. Merge: $\Gamma'_T = \Gamma'_F$ (both have it at $\bot$). Session complete.

7.2 Invalid Client: Missing Branch

$$\begin{aligned} e_{\text{bad}} = {} & \mathbf{let}\ \mathit{it} = \mathbf{new}\ \text{Iter}()\ \mathbf{in} \\ & \mathbf{let}\ r = \mathit{it}.\mathit{hasNext}()\ \mathbf{in} \\ & \mathit{it}.\mathit{next}() \quad \text{(no match!)} \end{aligned}$$

This fails: after $\mathit{hasNext}()$, it is at $s_\oplus$ (selection state). T-Call cannot apply at a selection state because $\mathit{enabledMethods}(s_\oplus) = \emptyset$. The client is *stuck*—it must use T-Match first.

Table 1: λ_S^2 correspondence with BICA Reborn's `TypestateChecker`.

Rule	**Behaviour**	**BICA construct**
T-Label	Enum constant, Γ unchanged	enum literal
T-Call	Returns enum at selection state	`advancePastSelections`
T-Match	Per-branch state advance	`CallNode.SelectionSwitch`
Merge	All branches reach same final state	`checkCallNodes` per branch

In BICA Reborn, this corresponds to the `advancePastSelections` mechanism: the checker tries to find a selection branch where `next` is enabled. In the TRUE branch it succeeds, but the FALSE branch has no `next` method, so the check depends on which branch the runtime takes. Without a match/switch, this is unsound.

8 Discussion and Conclusion

8.1 Implementation Correspondence

The key correspondence (Table 1): T-Match maps to `CallNode.SelectionSwitch` in BICA Reborn's `TypestateChecker`. The checker handles this in `checkCallNodes` (line 222): when it encounters a `SelectionSwitch` node, it forks, checks each branch independently against the corresponding selection successor state, and reports errors if any branch fails.

8.2 Orthogonality Breakdown

In λ_S^1, the Orthogonality Theorem states that Γ' is independent of *msig*. In λ_S^2, this no longer holds: the enum return value of a selection-triggering method determines *which* s_i the object enters, and therefore *how* Γ evolves inside each branch.

This is precisely the distinction between *data* returns (which are orthogonal) and *protocol* returns (which are not).

8.3 Preview: λ_S^3

λ_S^2 handles selection/branching but still restricts session types to finite trees. The next extension, λ_S^3, adds recursion ($\mu X.\, S \;/\; X$), enabling protocols like the full Iterator: $\mu X.\, \&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : X\},\ \mathit{FALSE} : \mathbf{end}\}\}$.

References

[1] Alexandre Zua Caldeira. λ_s^0 — a formal calculus for straight-line typestate checking (step 200a), 2026. Research note.